1. QUESTION 1

Finding the optimal parameters for k, L_{max}, and N_{min} requires analyzing the theoretical behavior of the HKM algorithm.

- k: Dictates tree breadth and depth. Higher values increase traversal complexity but allows for smaller, faster-to-scan leaves
- L_{max} & N_{min}: Governs early stopping of the tree height with inverse effects: decreasing L_{max} or increasing N_{min} halts the growth sooner. Their impact on query time is highly sensitive to the branching factor k and dataset size.

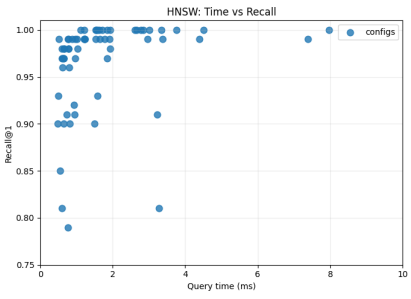
1.1. **Experiments.** To validate these trade-offs and identify optimal configurations, we designed a three phase study:

(1) Evaluate each hyperparameter independently from each other
(2) Evaluate parameters interactions by executing a Grid Search and identify promising regions of hyperparameter space
(3) Final random search using Optuna to identify probable parameters

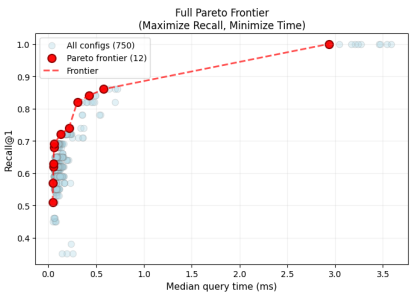
Due to the computational cost of the building phase, operations were conducted on the `SIFT10K` dataset. Phase 1 identified k and N_{min} as the primary hyperparameters. Notably, when observing the behavior of k, we found that as k approaches 100, the execution time spent computing distances between centroids rises significantly, while the time spent computing distances within leaves drops (see 1c) . This occurs because the number of clusters generated ($O(k^L)$) per split quickly outpaces the available data points. Phase 2 allowed us to better define the interactions between the parameters and narrow down our search space. Exploring the refined parameter space more broadly and efficiently using Optuna in Phase 3 we identified several candidate configurations (see 1b) , the one selected was $k = 27, L_{max} = 6, N_{min} = 132$ with average query time 0.0353, recall@1 of 0.55 and @5 of 0.84. Each configuration was better than the baseline brute-force search in terms of query times.

2. QUESTION 2

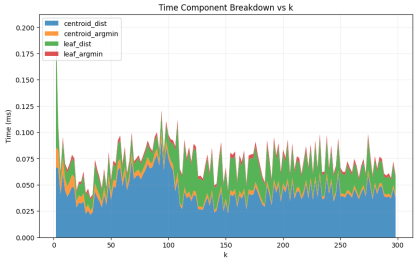
Searching the hyperparameter space of HNSW in M, $efSearch$, and $efConstruction$ shows that the algorithm maintains high recall with low query times, though it exhibits an increase in search time compared to lower-recall HKM executions (see plot 1a). HKM can fail when query points reside near the decision boundaries of the Voronoi partitions created by KMeans. This happends because the algorithm performs basic Greedy search, lacking the capability to backtrack. If at any point the algorithm moves to the wrong cluster, this becomes irreversible and will be an inevitable miss. In general this is due to the fact that the KMeans algorithm could have difficulty in building the right clusters in high dimensional spaces.



(A) Parameter Dependency



(B) Pareto Front



(C) K variations